Algorismes de cerca



Algorísmica

Grau d'Enginyeria Informàtica

Mireia Ribera, ribera@ub.edu



Algorisme de cerca: Conceptes clau

1. Cerca lineal
2. Cerca binària
3. Complexitat de la cerca binària
4. Cerca exhaustiva i intractable



Cercar

Cercar vol dir identificar un element que compleix cert criteri, per exemple:

- Cercar un element a una llista (el màxim, l'element que té el valor "x", el que compleix certa condició)
- Cercar un element en una llista *ordenada*
- Cercar un element en una estructura de dades complexa, com pot ser un graf o arbre.

Com l'ordenació, la cerca és una de les operacions més repetides a l'ordinador, i un pas previ a moltes altres operacions.

En aquesta assignatura ens centrarem en la cerca en llistes.



Cerca lineal (o)

L'algorisme que implementa la cerca amb una estratègia de força bruta s'anomena cerca seqüencial o cerca lineal. I té una complexitat O(n).

- Tenim una llista A d'n elements.

  Per ex. $A = [23, 27, 5, 90, 12, 44, 38, 84, 77]$
- Tenim un valor buscat, K, per ex. 38
- Recorrem la llista per trobar l'element amb el valor donat, començant pel primer element de la llista i fins que el trobem o arribem al final.
- Si trobem l'element retornem la seva posició i si no el trobem retornem -1.

0	1	2	3	4	5	6	7	8
23	27	5	90	12	44	38	84	77

0	1	2	3	4	5	6	7	8
23	27	5	90	12	44	38	84	77

0	1	2	3	4	5	6	7	8
23	27	5	90	12	44	38	84	77

0	1	2	3	4	5	6	7	8
23	27	5	90	12	44	38	84	77

0	1	2	3	4	5	6	7	8
23	27	5	90	12	44	38	84	77

0	1	2	3	4	5	6	7	8
23	27	5	90	12	44	38	84	77



Cerca lineal (1)

```python
def cerca_lineal(A:list[any],K:any):
  i:int=0
  while i < len(A) and A[i] != K:
    i += 1
  if i < len(A):
     return i
  else:
    return -1
```

Hi ha una iteració que, en el cas pitjor, passa per tots els elements, $O(n)$



Cerca lineal (2)

Podem fer una petita millora si afegim l'element buscat al final de la llista, però l'ordre de complexitat segueix sent el mateix, $O(n)$

```
def cerca_lineal(A:list[any],K:any):
  A.append(K)
  i:int=0
  while A[i] != K:   # sempre el troba i només ens cal una comparació
    i += 1
  if i < len(A)-1:
    return i
  else:
    return -1
```

Compte però, que estem alterant la llista original!



Cerca lineal (3)

Atenció!

🚨 Si la llista és ordenada hi ha algorismes més eficients que la cerca lineal!

Cercar dins una llista ordenada

Si la llista esta ordenada (un diccionari, el llistat de resultats de la loteria, etc.), podem cercar de manera més eficient!



Lotería Nacional

SORTEO EXTRAORDINARIO DE NAVIDAD

SORTEO NÚM. 102 DEL AÑO 2023

LISTA DE LOS NÚMEROS PREMIADOS EN CADA UNA DE LAS CIENTO OCHENTA Y CINCO SERIES CORRESPONDIENTES AL SORTEO CELEBRADO EN MADRID EL DÍA 22 DE DICIEMBRE DE 2023

Unidad

Decena

Centena

Tres mil

Seis mil

Nueve mil

Doce mil

Quince mil

Dieciocho mil

Veintiún mil

Veinticuatro mil

Treinta y tres mil

Treinta y seis mil

Veintisiete mil

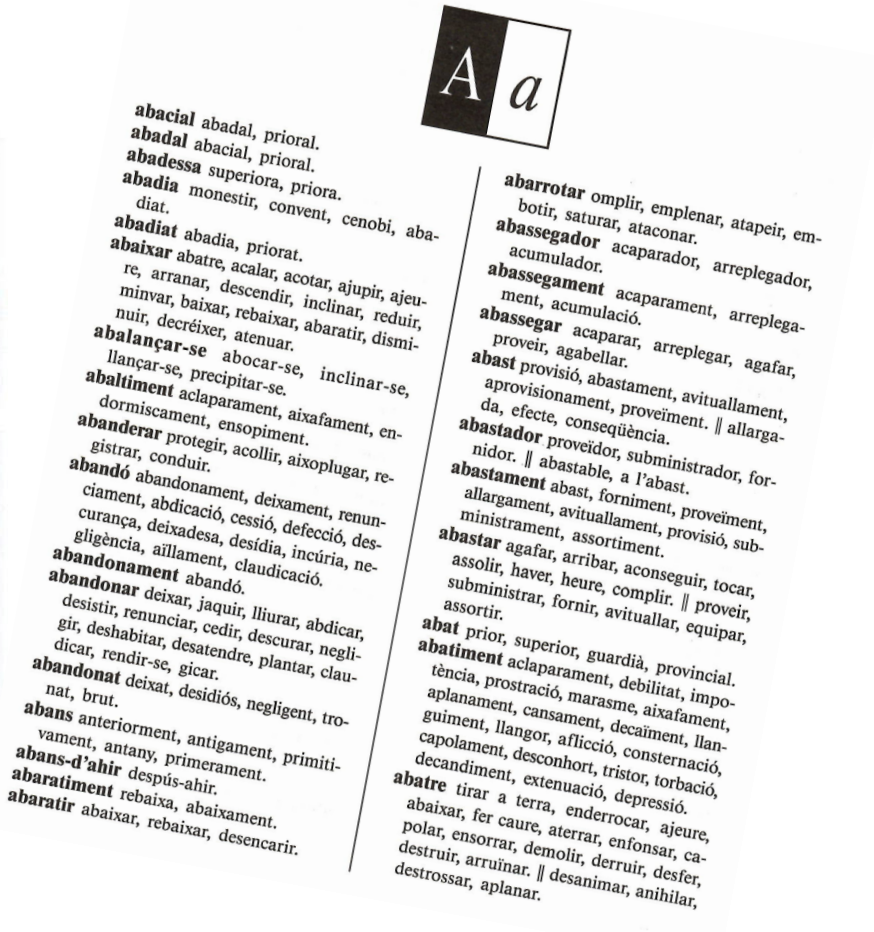
Cuarenta y dos mil

Cuarenta y cinco mil

Cuarenta y ocho mil

45353

60.000 EUROS



abacial abadal, prioral.
abadal abacial, prioral.
abadessa superiora, priora.
abadia monestir, convent, cenobi, abadiat.
abadiat abadia, priorat.
abaixar abatre, acalar, acotar, ajupir, ajeure, arranar, descendir, inclinar, reduir, minvar, baixar, rebaixar, abaratir, disminuir, decréixer, atenuar.
abalançar-se abocar-se, inclinar-se, llançar-se, precipitar-se.
abaltiment aclaparament, aixafament, endormiscament, ensopiment.
abanderar protegir, acollir, aixoplugar, registrar, conduir.
abandó abandonament, deixament, renunciament, abdicació, cessió, defecció, desconfiança, deixadesa, desídia, incúria, negligència, aïllament, claudicació.
abandonament abandó.
abandonar deixar, jaquir, lliurar, abdicar, desistir, renunciar, cedir, descurar, negligir, deshabitar, desatendre, plantar, clausicar, rendir-se, gicar.
abandonat deixat, desidiós, negligent, trobat, brut.
abans anteriorment, antigament, primitivament, antany, primerament.
abans-d'ahir despús-ahir.
abaratiment rebaixa, abaixament.
abaratir abaixar, rebaixar, desencarir.

abarrotar omplir, emplenar, atapeir, embotir, saturar, ataconar.
abassegador acaparador, arreplegador, acumulador.
abassegament acaparament, arreplegament, acumulació.
abassegar acaparar, arreplegar, agafar, proveir, agabellar.
abast provisió, abastament, avituallament, aprovisionament, proveïment. ‖ allargada, efecte, conseqüència.
abastador proveïdor, subministrador, fornidor. ‖ abastable, a l'abast.
abastament abast, forniment, proveïment, allargament, avituallament, provisió, subministrament, assortiment.
abastar agafar, arribar, aconseguir, tocar, assolir, haver, heure, complir. ‖ proveir, subministrar, fornir, avituallar, equipar, assortir.
abat prior, superior, guardià, provincial.
abatiment aclaparament, debilitat, impotència, prostració, marasme, aixafament, aplanament, cansament, decaïment, llanguiment, llangor, aflicció, consternació, capolament, desconhort, tristor, torbació, decandiment, extenuació, depressió.
abatre tirar a terra, enderrocar, ajeure, abaixar, fer caure, aterrar, enfonsar, capolar, ensorrar, demolir, derruir, desfer, destruir, arruïnar. ‖ desanimar, anihilar, destrossar, aplanar.



Cerca binària

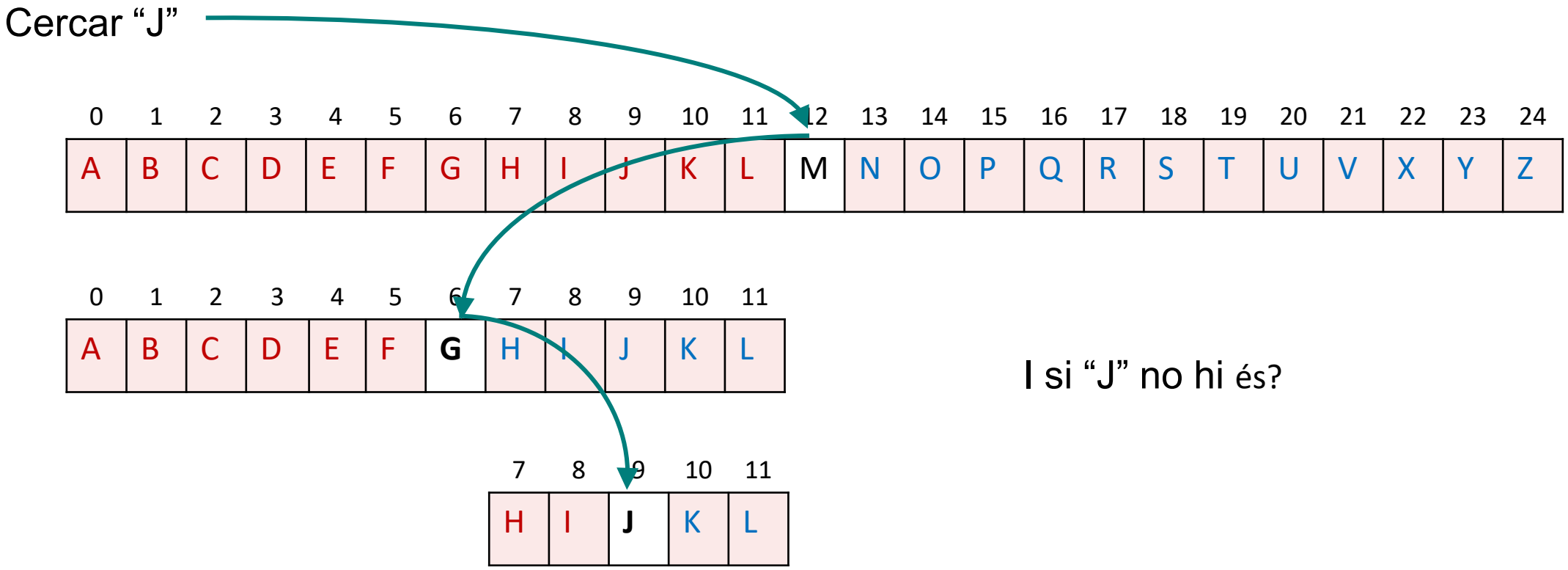
La cerca binària consisteix en comparar l'element a buscar, K, amb l'element central de la llista: si hi ha correspondència ja l'hem trobat, altrament, busquem K a la subllista de la dreta o de l'esquerra segons correspongui.

$$A[0], \dots, A[s-1], A[s], A[s+1], \dots, A[n-1]$$

Buscarem a l'esquerra si $K < A[s]$ | K | Buscarem a la dreta si $K > A[s]$



Cerca binària: exemple 1



I si “J” no hi és?

Quan implementem en codi, en comptes de “tallar” la llista, podem indicar la finestra d’interès amb dos punters, inici i final.



Cerca binària, codi recursiu tallant la llista

```python
def cerca_binaria_rec(A:list[any],K:any):
    mig = len(A) // 2
    if A[mig] > K:
        return cerca_binaria_rec(A[0:mig], K)
    elif A[mig] < K:
        return cerca_binaria_rec(A[mig+1:len(A)], K)+mig+1 # sumo el desplaçament
    else:
        return mig
    return -1
```



Cerca binària: exemple 2, amb finestra

70

	0	1	2	3	4	5	6	7	8	9	10	11	12
	3	14	27	31	39	42	55	70	74	81	85	93	98
Iteració 1	Inici						Mig						Final
Iteració 2								inici			Mig		Final
Iteració 3								Inici	Mig	Final			
Iteració 4								Inici, Mig, Final					

- A la primera iteració comparem 55 amb 70. Com que 55 és menor seguim amb la cerca per la dreta.
- A la segona iteració comparem 85 amb 70. Com que 85 és més gran seguim amb la cerca per la dreta.
- A la tercera iteració comparem 74 amb 70. Com que 74 és més gran seguim amb la cerca per l'esquerra.
- A la quarta iteració comparem 70 amb 70. L'hem trobat i per tant, acabem!



Cerca binària, codi iteratiu equivalent, amb finestra

```
def cerca_binaria(A:list[any],K:any):
    inici = 0
    final = len(A)-1
    while inici <= final:
        mig = (inici + final) // 2
        if A[mig] > K:
            final = mig - 1
        elif A[mig] < K:
            inici = mig + 1
        else:
            return mig
    return -1
```



Cerca binària, complexitat

Si analitzem el problema amb el teorema Màster, veiem que a cada iteració dividim l'entrada en 2 (b), i que fem 1 única crida recursiva (a), i que el cost de preparar i combinar les respostes és constant (d) [si usem finestres].

Per tant estaríem en el cas 2: $T(n) = O\left(n^d log \quad \mathrm{n}\right)$; $a = b^d$, del teorema màster amb $d = 0$ i per tant, la complexitat serà de $O(log_2 n)$

Per a una llista de 1.000.000 elements només hauríem de fer 20 comparacions!



Cerca binària, complexitat, reflexions finals

Si tenim una llista desordenada de mida n i només hem de buscar un element (o pocs), apliquem una cerca lineal. Però si hem de fer moltes cerques (de l'ordre d'n), és més eficient ordenar-la primer i fer cerca binària dels elements després!



Notebook de suport

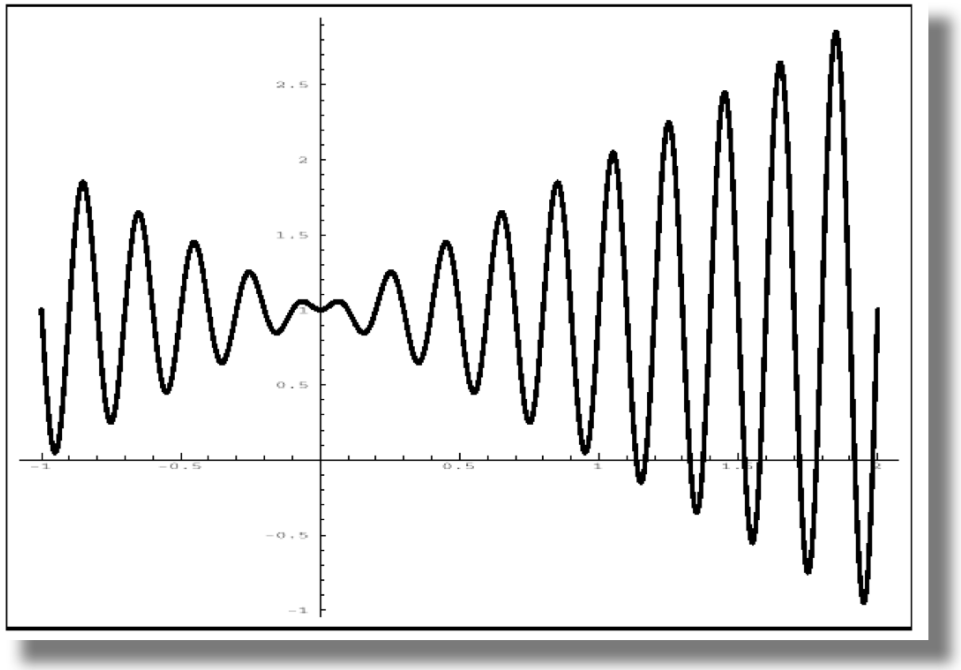
Podeu consultar tots els codis d'aquest tema a: Tema7Cercar-Codi.ipynb



Cerca exhaustiva

Algunes cerques no són tan senzilles com buscar un element o el màxim dins una llista acotada. Per exemple, en la funció següent, com busquem el màxim? quin tipus de cerca hi podem aplicar?

Podríem aplicar cerca exhaustiva (de complexitat O(n)), on n és el nombre de x de la funció que considerem.



Cerca exhaustiva, exemple



```python
import math

def func1d(x:int): # funció multimodal de la que buscarem max
    y: float = x * math.sin(10*math.pi*(x))+1.0
    return y

def frange(start: float, stop: float, step: float):
                                # definim un range sobre floats per conveniència
    current: float = start
    while current < stop:
        yield current           # yield proveeix un resultat i segueix
        current += step

def cerca_exhaustiva_func1d():   # la funció de cerca
    x:int = -1                   # valor inicial d'x
    maxim:float = func1d(x)      # valor inicial d'y
    for i in frange(-1.0, 2.0, 0.01):
        if func1d(i)>maxim:
            maxim = func1d(i)
            x = i
    print(x, maxim)
```

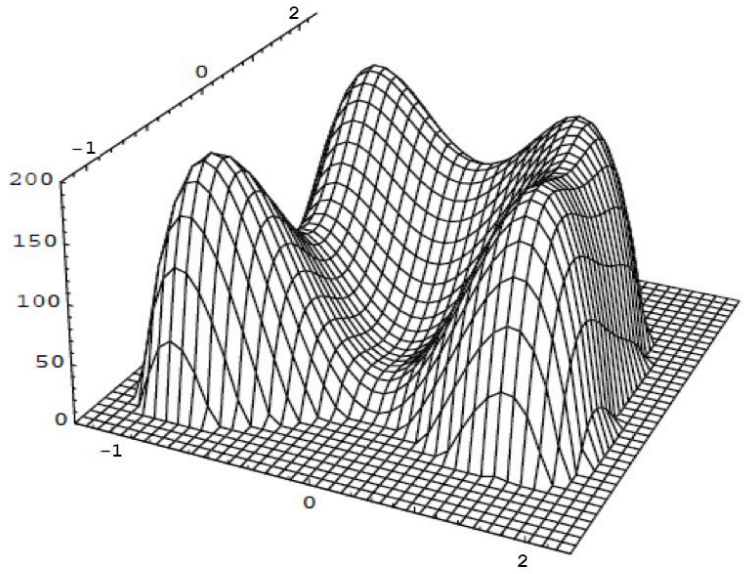


Cerca exhaustiva = intractable

Si el nombre de punts a mostrejar és molt gran, ni tan sols la cerca exhaustiva és una solució.

En aquest cas si analitzem els punts amb una precisió de 0.01 analitzaríem 300x300 punts.

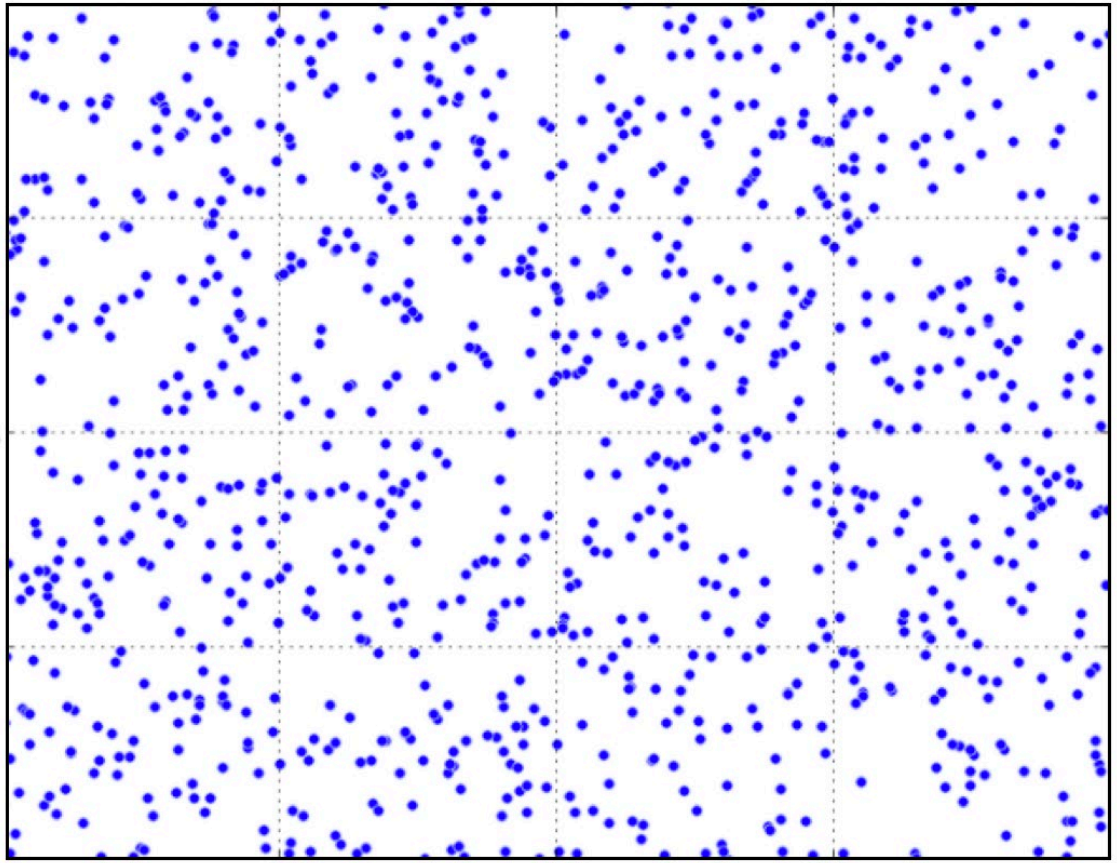
Si necessitéssim una precisió 0.000001 analitzaríem 3000000x3000000 punts.





Cerca aleatòria?

Podríem plantejar-nos d'anar generant nombres aleatòriament dins el rang de valors de x, calcular-ne la funció i triar el màxim.





Cerca aleatòria, exemple

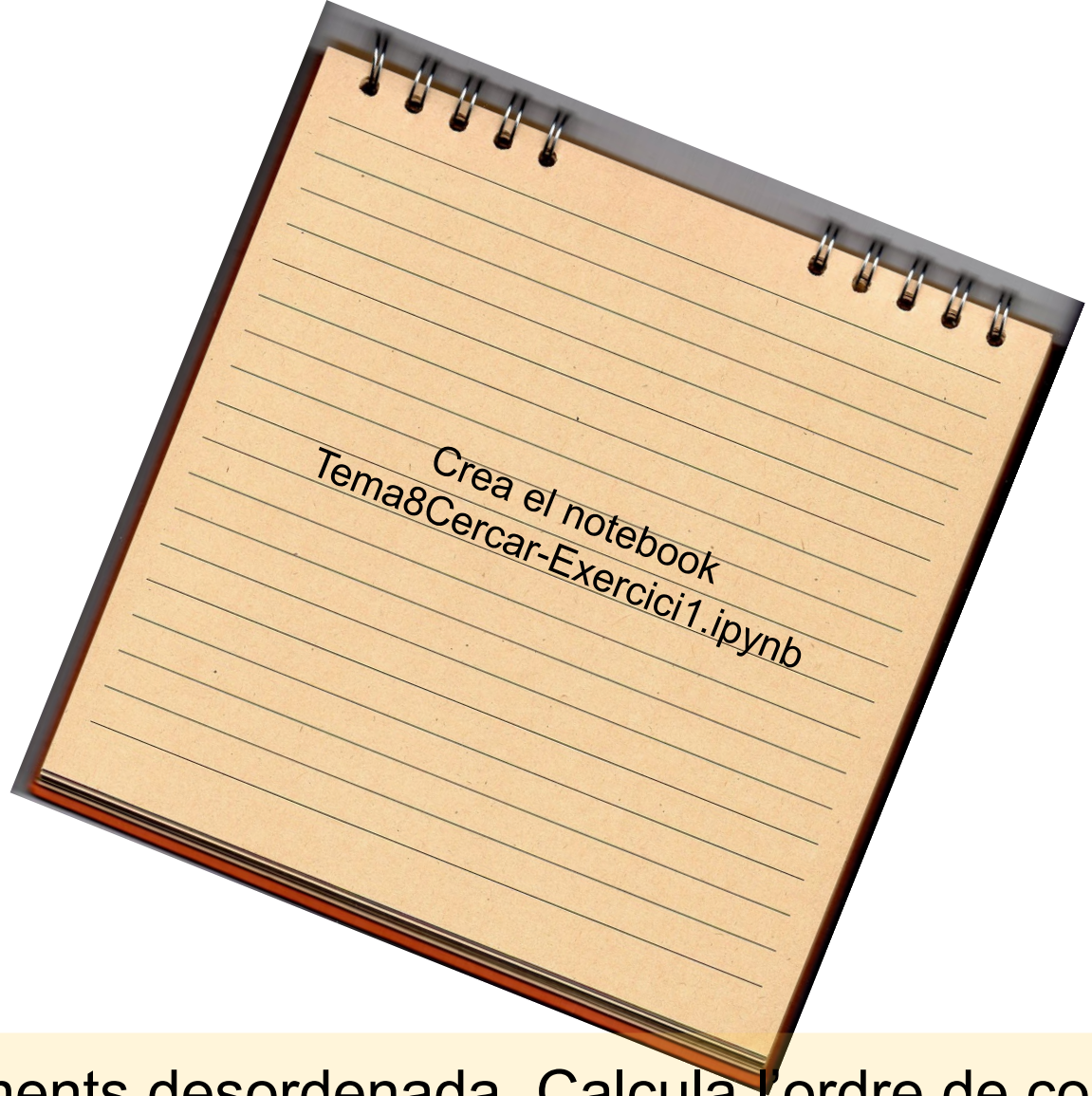
```python
import random


def cerca_aleatoria_func1d():
    x = -1  #valor inicial d'x
    maxim=func1d(x)  # valor inicial d'y
    for i in range(1000): #provem 1000 valors aleatòriament
      xtemp = (random.random()*3.0)-1.0   # valors entre -1 i 2
      if func1d(xtemp)>maxim:
          maxim=func1d(xtemp)
          x=xtemp
    return x, maxim
```

En general la cerca totalment aleatòria **no** és una bona solució.



Ara et toca...



Tens una llista de 30 elements desordenada. Calcula l'ordre de complexitat en cerca lineal i en cerca binària i decideix a partir de quantes cerques valdria la pena ordenar la llista. Fes un algoritme que ho calculi per un nombre donat d'elements.

Gràcies i recapitulació

MIREIA RIBERA | ACCESSIBILITAT DIGITAL
| EXPERIÈNCIA D'USUARI
| VISUALITZACIÓ DE DADES

UNIVERSITAT DE BARCELONA



Recapitulació

🔍➡️ La cerca lineal, amb un cost d'O(n) és assumible si una llista està desordenada i no fem moltes cerques.

🔎➗ Quan fem moltes cerques o partim d'una llista ordenada, l'algorisme de cerca binària és òptim.

🤯⚠️ No hem vist cap bona solució per a problemes de cerca complexos.